

PREDICTING ANNUAL RAINFALL

COURSE: MACHINE LEARNING

TEAM : GEEK LEARNERS

PROJECT ID:18MLSNCSB25

II year CSE B

SAI SANJAY NAGARUR – 17BD1A0531

SANNITHI SHARAN KUMAR -17BD1A053G

GALI VENKATA SAI NEEL LOHIT-17BD1A052H

GUIDE: SWETHA

Keshav Memorial Institute of Technology, Hyderabad

School of New and Emerging Technology (SONET)

C E R T I F C A T E

This is to certify that Mr GALI VENKATA SAI NEEL LOHIT bearing Hall Ticket No 17BD1A052H has successfully completed the project titled: PREDICTING ANNUAL RAINFALL under the Data Science Stream and submitted a satisfactory report.

Coordinator

Guide

Director

ACKNOWLEDGEMENT

The success and final outcome of this project required a lot of guidance and assistance from many people and I am extremely privileged to have got this all along the completion of my project. All that I have done is only due to such supervision and assistance and I would not forget to thank them.

I respect and thank **Prof. Neil gogte**, for providing us an opportunity to do the project work in Machine Learning and giving us all support and guidance which made us complete the project duly. I am extremely thankful to him for providing such a nice support and guidance, although he had busy schedule managing the corporate affairs.

I owe my deep gratitude to our project guide P V N Balarama Murthy sir, who took keen interest on our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

I would not forget to remember Mrs.Deepa and Mr.Aditya Godse, of kmit for their encouragement and more over for their timely support and guidance till the completion of our project work.

I heartily thank our internal project guides, Ajeet k.Jain, Gnanesh Aggadi ,Venkatesh Kethineni for their guidance and suggestions during this project work.

I am thankful to and fortunate enough to get constant encouragement, support and guidance from all Teaching staffs of CSE Department which helped us in successfully completing our project work. Also, I would like to extend our sincere esteems to all staff in laboratory for their timely support.

[Name Surname]

TABLE OF CONTENTS

1.ABSTRACT	**7**
2.INTRODUCTION	**8**
3.PROBLEM STATEMENT AND METHODOLOGY	**10**
4. HARDWARE AND SOFTWARE REQUIRED	**10**
5. LIST OF LIBRARIES AND PACKAGES USED	**10**
6. DATA PREPROCESSING	**16**
7.VISUALIZING DATA	**23**
8.TRAINING THE MODEL	**26**
9.TESTING THE MODEL	**27**
10.JUSTIFICATION OF FITTED MODEL	**30**
11.CLASSIFICATION REPORT	**31**
12.ROC CURVE	**34**
13.DISCUSSION ON RESULTS	**37**
14. LIST OF REFERENCES	**39**

ABSTRACT

DATASET: RAINFALL DATSET

- Rainfall is crucial for the economic welfare of the country.so its important to predict the rainfall
- In our country and be prepared to face the situations. In this project we take a sample data set containing
- The rainfall cast during a particular period and we use Machine Learning to predict the annual rainfall.

In this project we may use scikitlearn features such as linear regression ,classification ,logistic regression, pyplot,mse,rmse..etc.

DESCRIPTION: Dataset contains the information regarding rainfall cast during the years 1991

in the states of India. Also provides data regarding annual rainfall and periodical rainfall in the respective states.

What can be predicted?

- To predict states which receive heavy and least rainfalls
- To predict states which have good annual rainfall.
- To predict areas which may face water scarcity/drought.
- To safeguard areas which may get floods which may receive heavy rainfalls

Source link:
https://www.kaggle.com/rajanand/rainfall-in-india/downloads/rainfall%20in%20india%201901-2015.csv/2

INTRODUCTION

DOMAIN ANALYSIS:

Rainfall plays an important role in agriculture so early prediction of rainfall is necessary for the better economic growth of our country. Rainfall prediction has been the one of the most challenging issues around the world in last year. Agriculture is the backbone of Indian economy. Irrigation facility is still not so good in India and most of agriculture depends upon the rain. A good rainfall result in the occurrence of a dry period for a long time or heavy rain both affect the crop yield as well as the economy of country, so due to that early prediction of rainfall is very crucial.

A wide range of rainfall forecast methods are employed in weather prediction at regional and national levels. Fundamentally there are two approaches to predict Rainfall. They are Empirical and Dynamical Methods.

The **Empirical approach** is based on analysis of past historical data of weather and its relationship to a variety of atmospheric variables over different parts of India. The most widely use empirical approaches used for climate prediction are Regression**,** artificial neural network, fuzzy logic and group method of data handling.

The **dynamical approach**, predictions are generated by physical models based on system of equations that predict the future Rainfall. The forecasting of weather by computer using equations are known as numerical weather prediction. To predict the weather by numeric means, meteorologist has develop atmospheric models that approximate the change in temperature, pressure etc using mathematical equations.

PROPOSED METHOD OF RAINFALL PREDICTION:

MULTIPLE LINEAR REGRESSION:

Regression is a statistical measure that attempts to determine the strength of the relationship between one dependent variable usually denoted by Y and a series of other changing variables known as independent variables. Regression model which contain more than two predictor variables are called Multiple Regression Model. Multiple regression model is of the form:

Y=b0+b1x1 +b2x2 +b3x3+ b4x4+...e

where b0,b1,b2,b3,b4 are regression coefficients e is unexplained portion of dependent variable with zero mean and constant variance. Multiple regression fits a model to predict a dependent (Y) variable from two or more independent (X) variables

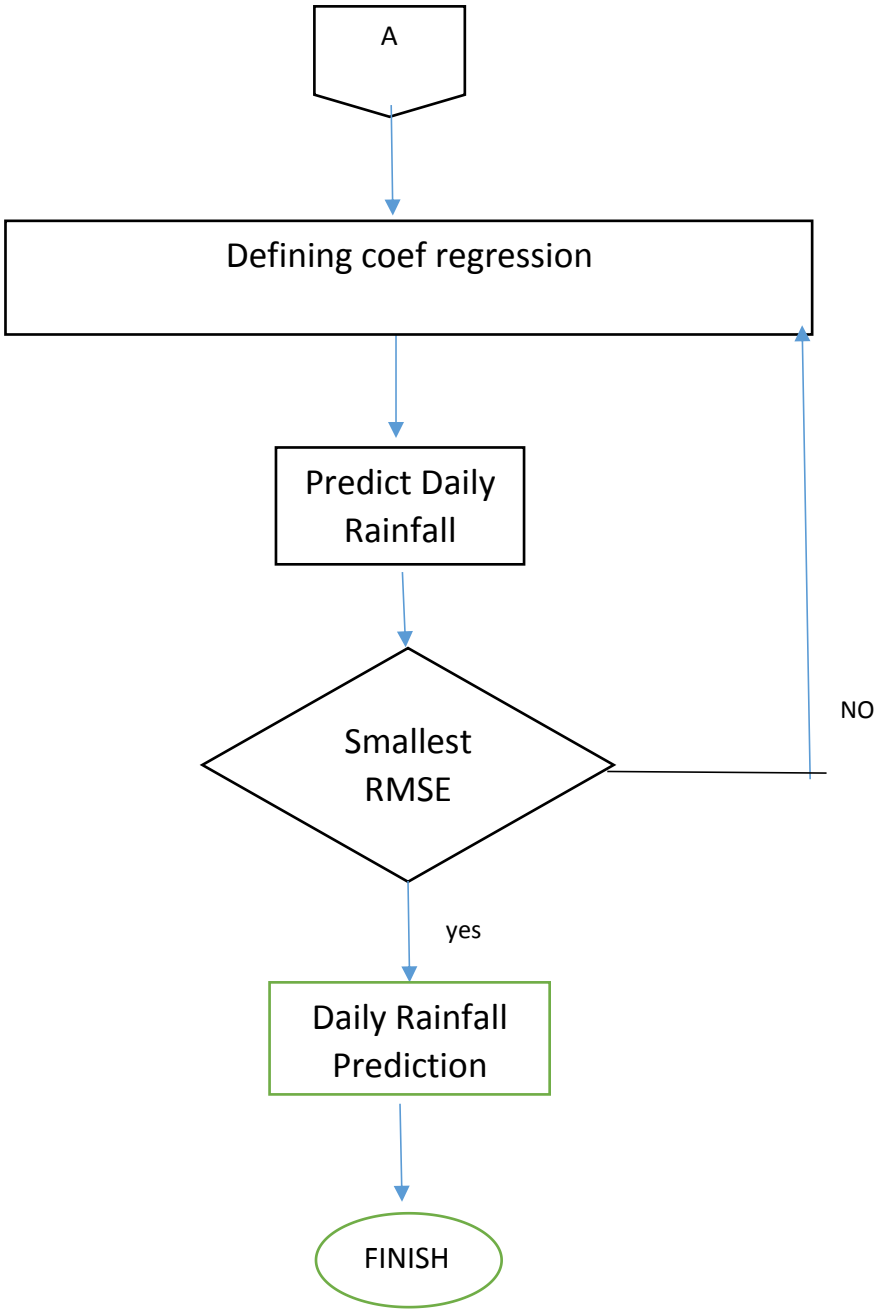


Figure 1. Double Regression Steps

ANALYSING THE DATASET: The dataset constitutes the amount of rainfall received by 29 states of India in mm from the year 1901-2015. There are 4116 entries

in the dataset (rows) and 19 attributes(columns) namely annual rainfall **-[‘ANNUAL’] , [‘SUBDIVISION’] ,[’YEAR’],[‘JAN’,‘FEB’,‘MAR’….’DEC’],[’JAN-FEB’,‘MAR-MAY’,’JUN-SEP’,’OCT-DEC’]**.

PROBLEM STATEMENT:

To Predict the **‘annual rainfall’** ([‘ANNUAL’] attribute) in the 29 states of India between 1901-2015 from the **‘monthly rainfall’** ([‘JAN-FEB’,…’OCT-DEC’] attribute) by using *multiple linear regression.*

PROCEDURE:

1.DATA PRE-PROCESSING:

2.VISUALIZING DATA

3.TRAIN THE DATA

4.TEST THE MODEL USONG PERFORMNACE MEASURES.

HARDWARE AND SOFTWARE REQUIRED:

HARDWARE:

Processor: Intel(R) Core (TM) i7-7700HQ CPU @ 2.80GHz (8 CPUs), ~2.8GHz

Memory: 8192MB RAM

OS: Windows 10 Home 64-bit (10.0, Build 17134)

OS Memory: 8058MB RAM.

SOFTWARE TOOLS:

Python version 3.6.5, Anaconda navigator version 1.9.2 , Numpy version , Pandas version sci-kit learn version

LIST OF LIBRARIES AND PACKAGES USED:

NumPy, Pandas, Sci-kit learn, matplotlib, seaborn.

```
import pandas as pd
import numpy as np
import math
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error,r2_score,mean_absol
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn import preprocessing
```

NUMPY: multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays. NumPy is open-source software and has many contributors.

NumPy is the fundamental package for scientific computing with Python. It contains among other things:

Library features

- a powerful N-dimensional array object
- sophisticated (broadcasting) functions
- tools for integrating C/C++ and Fortran code
- useful linear algebra, Fourier transform, and random number capabilities

PANDAS: pandas is a software library written for the Python programming language for data manipulation and analysis. In particular, it offers data structures and operations for manipulating numerical tables and time series. It is a free software.

Library features

- DataFrame object for data manipulation with integrated indexing.
- Tools for reading and writing data between in-memory data structures and different file formats.
- Data alignment and integrated handling of missing data.
- Reshaping and pivoting of data sets.
- Label-based slicing, fancy indexing, and subsetting of large data sets.
- Data structure column insertion and deletion.
- Group by engine allowing split-apply-combine operations on data sets.

- Data set merging and joining.
- Hierarchical axis indexing to work with high-dimensional data in a lower-dimensional data structure.
- Time series-functionality: Date range generation[4] and frequency conversion, moving window statistics, moving window linear regressions, date shifting and lagging.
- Provides data filtration.

SCI-KIT LEARN: Scikit-learn (formerly scikits.learn) is a free software machine learning library for the Python programming language.[3] It features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, *k*-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

Library features

1.Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.

Algorithms: SVM, nearest neighbors,random forest, …

2. Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, ridge regression, Lasso, …

3. Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering,mean-shift, …

4. Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: PCA, feature selection, non-negative matrix factorization.

5. Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning

Modules: grid search, cross validation,metrics.

6. Preprocessing

Feature extraction and normalization.

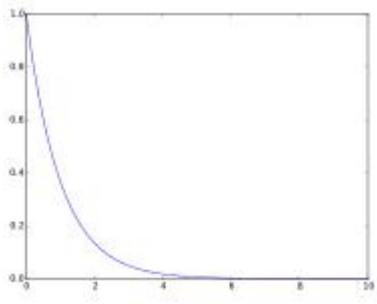
Application: Transforming input data such as text for use with machine learning algorithms.

Modules: preprocessing, feature extraction.

MATPLOTLIB: matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-orientedAPI for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK+. There is also a procedural "pylab" interface based on a state machine (like OpenGL), designed to closely resemble that of MATLAB, though its use is discouraged. SciPy makes use of matplotlib.

Library Features

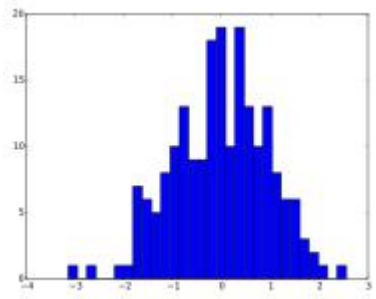
Line plot



```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> a = np.linspace(0, 10, 100)
>>> b = np.exp(-a)
>>> plt.plot(a, b)
```

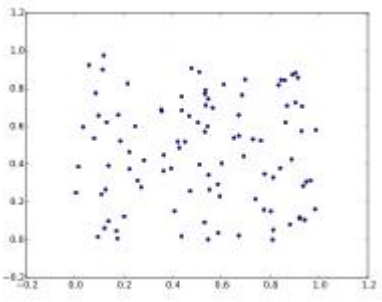
```
>>> plt.show()
```

Histogram



```
>>> import matplotlib.pyplot as plt
>>> from numpy.random import normal,rand
>>> x = normal(size=200)
>>> plt.hist(x, bins=30)
>>> plt.show()
```

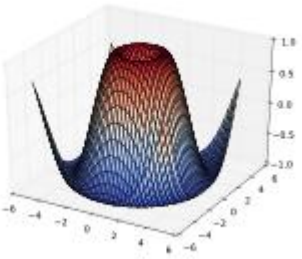
Scatter plot



```
>>> import matplotlib.pyplot as plt
>>> from numpy.random import rand
>>> a = rand(100)
>>> b = rand(100)
```

```
>>> plt.scatter(a, b)
>>> plt.show()
```

3D plot



```
>>> from matplotlib import cm
>>> from mpl_toolkits.mplot3d import Axes3D
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig = plt.figure()
>>> ax = fig.gca(projection='3d')
>>> X = np.arange(-5, 5, 0.25)
>>> Y = np.arange(-5, 5, 0.25)
>>> X, Y = np.meshgrid(X, Y)
>>> R = np.sqrt(X**2 + Y**2)
>>> Z = np.sin(R)
>>> surf = ax.plot_surface(X, Y, Z, rstride=1, cstride=1,
cmap=cm.coolwarm)
>>> plt.show()
```

DATA PRE-PROCESSING

Real world data are generally:

- **INCOMPLETE**: lacking attribute values, lacking certain attributes of interest, or
- **NOISY**: containing errors or outliers.
- **INCONSISTENT**: containing discrepancies in codes or names.

TASKS IN DATA PRE-PROCESSING:

- Data preprocessing is an important step of solving every machine learning problem.

- Most of the datasets used with machine learning problems need to be processed/cleaned/transformed so that a machine learning algorithm can be trained on it.

- Most commonly used pre-processing techniques are very few like-missing value imputation, encoding categorical variables, scaling etc.

The type of data we are handling is **Numerical Data:**

HANDLING NUMERICAL DATA:

- Too many nulls- when most (over 60% to 70%) of the values in a column are null, its better to drop the column.
- Same values/skew- Sometimes a majority of values in a column might be same values with very few different values. We need to check if the occurrence of such values is due to a skew in dataset or is it natural for that dataset. If its skewed the dataset should be resampled. If its not s skew and the values occur naturally in that way, it's better to drop the column.
- **Data types-**check the data types of the columns, particularly the data columns and type cast appropriately.
- **Missing value imputation –** usually median is used with numeric columns and mode with non-numeric columns.
- **When column doesn't have missing values-**its possible that the column doesn't have any null values in the train dataset but very possible that it might have null values in the test dataset. Hence its important to review the columns/data and perform missing value imputation of all columns that may have missing values

TECHNIQUES:

1.REMOVING NULL VALUES:

```
data=pd.read_csv("C:\\Users\\Nagarur\\Documents\\MachineLearning\\rainfall.cs
print("Data heads:")
print(data.head())
print("Null values in the dataset before preprocessing:")
print(data.isnull().sum())
print("Filling null values with mean of that particular column")
data=data.fillna(np.mean(data))
print("Mean of data:")
print(np.mean(data))
print("Null values in the dataset after preprocessing:")
print(data.isnull().sum())
print("\n\nShape: ",data.shape)
```

```
Null values in the dataset before pre-processing:

SUBDIVISION     0
YEAR            0
JAN             4
FEB             3
MAR             6
APR             4
MAY             3
JUN             5
JUL             7
AUG             4
SEP             6
OCT             7
NOV            11
```

```
DEC          10
ANNUAL        26
Jan-Feb        6
Mar-May        9
Jun-Sep       10
Oct-Dec       13
```

Filling null values with mean of that particular column:

```
Mean of data:
YEAR      1958.218659
JAN         18.957320
FEB         21.805325
MAR         27.359197
APR         43.127432
MAY         85.745417
JUN        230.234444
JUL        347.214334
AUG        290.263497
SEP        197.361922
OCT         95.507009
NOV         39.866163
DEC         18.870580
ANNUAL    1411.008900
Jan-Feb     40.747786
Mar-May    155.901753
Jun-Sep   1064.724769
Oct-Dec    154.100487
```

Null values in the dataset after preprocessing:

```
SUBDIVISION   0
YEAR          0
JAN           0
FEB           0
```

```
MAR           0
APR           0
MAY           0
JUN           0
JUL           0
AUG           0
SEP           0
OCT           0
NOV           0
DEC           0
ANNUAL        0
Jan-Feb       0
Mar-May       0
Jun-Sep       0
Oct-Dec       0
dtype: int64


Shape:  (4116, 19)
```

2. CHECKING THE DATATYPES OF COLUMNS:

```
print("Info:")
print(data.info())
```

```
Data columns (total 19 columns):
SUBDIVISION    4116 non-null object
YEAR           4116 non-null int64
JAN            4116 non-null float64
FEB            4116 non-null float64
MAR            4116 non-null float64
APR            4116 non-null float64
MAY            4116 non-null float64
JUN            4116 non-null float64
JUL            4116 non-null float64
AUG            4116 non-null float64
```

```
SEP           4116 non-null float64
OCT           4116 non-null float64
NOV           4116 non-null float64
DEC           4116 non-null float64
ANNUAL         4116 non-null float64
Jan-Feb        4116 non-null float64
Mar-May        4116 non-null float64
Jun-Sep        4116 non-null float64
Oct-Dec        4116 non-null float64
dtypes: float64(17), int64(1), object(1)
```

3.COVARIANCE AND CORRELATION:

```
print("Co-Variance =",data.cov())
print("Co-Relation =",data.corr())
```

```
Co-Variance =                YEAR          JAN          FEB          MAR          APR  \
YEAR     1098.319127   -62.525455   -26.333846    31.608776    17.985597
JAN       -62.525455  1126.880700   549.299548   627.375837   475.344306
FEB       -26.333846   549.299548  1288.551221   974.300386   892.111938
MAR        31.608776   627.375837   974.300386  2201.972143  1766.214414
APR        17.985597   475.344306   892.111938  1766.214414  4596.594854
MAY        14.668989   535.377197   896.648721  2094.072720  5433.121872
JUN      -105.646427  -265.334667   283.409286  1819.002965  7239.347234
JUL      -144.815256  -465.128448   156.524886  1227.377060  4883.946095
AUG        40.268753    75.536029   488.385320  1195.021166  3275.971621
SEP       -29.893921   110.216877   388.537697  1133.597551  3507.500523
OCT         7.922814    41.252990   -16.324506   401.044464  2478.380760
NOV       -42.624271   154.517515   -57.342083    28.337749   767.024409
DEC       -26.804831   311.215537   200.895960   270.309451   379.022457
ANNUAL   -239.492392  3182.200682  5831.636967  13551.372253  34892.913572
Jan-Feb   -87.619668  1675.112786  1837.200844  1601.188055  1367.102182
Mar-May    70.841300  1632.972340  2754.690073  6057.993672  11751.856167
Jun-Sep  -220.389375  -537.708253  1291.239831  5354.542547  18800.990715
Oct-Dec   -56.011703   507.405040   130.324910   702.446620  3611.891111
```

4.CORRELATIONAL COLUMNS:

```
corr_cols=data.corr()['ANNUAL'].sort_values()[::-1]
print("Index of correlation columns:",corr_cols.index)
```

```
Index of correlation columns: Index(['ANNUAL', 'Jun-Sep', 'JUN', 'JUL', 'AUG', 'SEP', 'MAY', 'Mar-May',
       'OCT', 'APR', 'Oct-Dec', 'MAR', 'NOV', 'DEC', 'FEB', 'Jan-Feb', 'JAN',
       'YEAR'],
      dtype='object')
```

3.GROUPING BY SUBDIVISION:

```
print("Group by:")
data.groupby('SUBDIVISION').size()
```

```
SUBDIVISION
ANDAMAN & NICOBAR ISLANDS           110
ARUNACHAL PRADESH                    97
ASSAM & MEGHALAYA                   115
BIHAR                               115
CHHATTISGARH                        115
COASTAL ANDHRA PRADESH              115
COASTAL KARNATAKA                   115
EAST MADHYA PRADESH                 115
EAST RAJASTHAN                      115
EAST UTTAR PRADESH                  115
GANGETIC WEST BENGAL                115
GUJARAT REGION                      115
HARYANA DELHI & CHANDIGARH          115
HIMACHAL PRADESH                    115
JAMMU & KASHMIR                     115
JHARKHAND                           115
KERALA                              115
KONKAN & GOA                        115
LAKSHADWEEP                         114
MADHYA MAHARASHTRA                  115
MATATHWADA                          115
NAGA MANI MIZO TRIPURA              115
NORTH INTERIOR KARNATAKA            115
ORISSA                              115
PUNJAB                              115
RAYALSEEMA                          115
SAURASHTRA & KUTCH                  115
SOUTH INTERIOR KARNATAKA            115
SUB HIMALAYAN WEST BENGAL & SIKKIM  115
TAMIL NADU                          115
```

```
TELANGANA                          115
UTTARAKHAND                        115
VIDARBHA                           115
WEST MADHYA PRADESH                115
WEST RAJASTHAN                     115
WEST UTTAR PRADESH                 115
```

VISUALIZING THE DATA

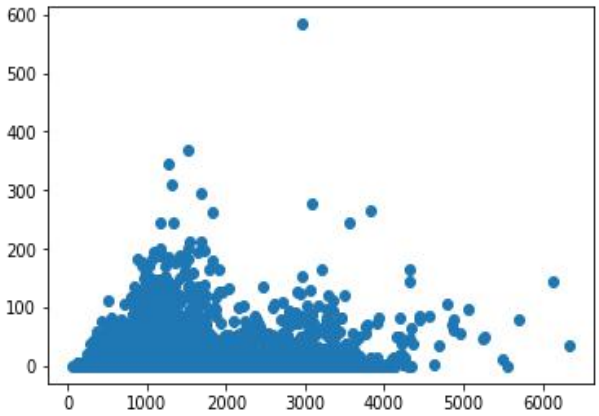
We have previously learnt about the tools of the matplotlib library like **histograms, box plots, scatter plots** used for visualizing data. Now we are going to apply those tools.

TECHNIQUES:

1.SCATTER PLOT:

```
print("Scatter plot of annual and january attributes")
plt.scatter(data.ANNUAL,data.JAN)
```

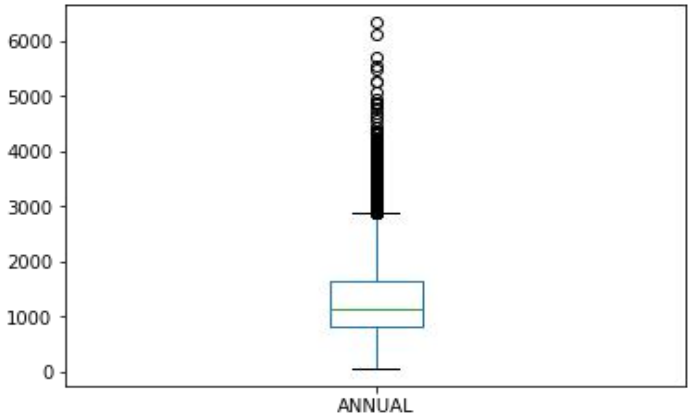
Scatter plot of annual and january attributes:



2.BOX PLOT:

```
print("Box Plot of annual rainfall data in years 1901-2015")
data['ANNUAL'].plot(kind='box',sharex=False,sharey=False)
```

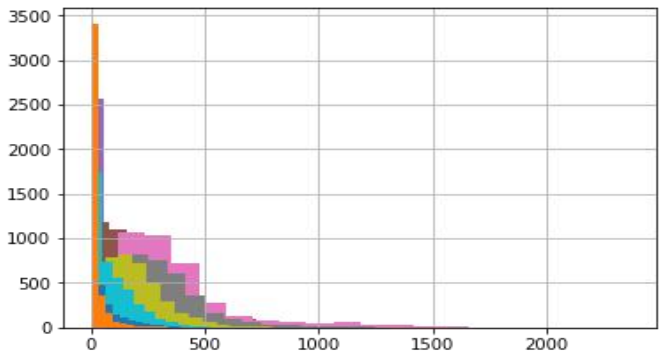
Box Plot of annual rainfall data in years 1901-2015:



3.HISTOGRAM:

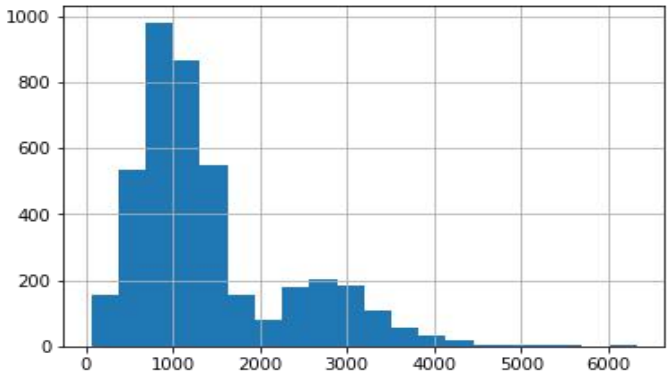
```
print("Histograms showing the data from attributes (JANUARY to DECEMBER) of the years 1901-2015:")
data['JAN'].hist(bins=20)
data['FEB'].hist(bins=20)
data['MAR'].hist(bins=20)
data['APR'].hist(bins=20)
data['MAY'].hist(bins=20)
data['JUN'].hist(bins=20)
data['JUL'].hist(bins=20)
data['AUG'].hist(bins=20)
data['SEP'].hist(bins=20)
data['OCT'].hist(bins=20)
data['NOV'].hist(bins=20)
data['DEC'].hist(bins=20)
```

Histograms showing the data from attributes (JANUARY to DECEMBER) of the years 1901-2015:




```
print("Histogram showing the annual rainfall of the all states:")
data['ANNUAL'].hist(bins=20)
```

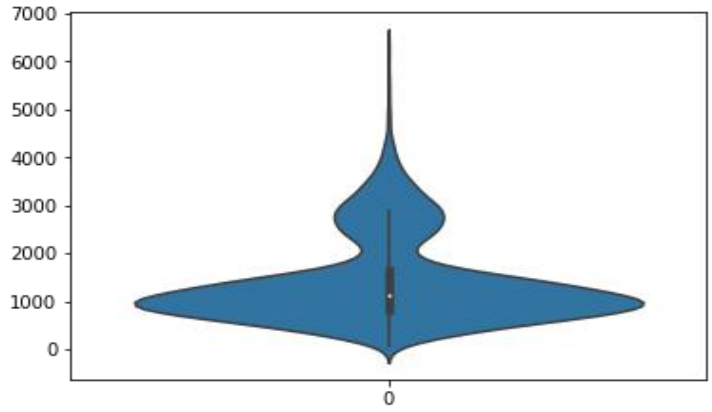
Histogram showing the annual rainfall of the all states:



3.VIOLIN PLOT:

```
print("Violin plot of the ANNUAL attribute :-")
sb.violinplot(data=data['ANNUAL'])
```

Violin plot of the ANNUAL attribute:



TRAINING THE MODEL

Here we have used *multiple linear regression* to predict the annual Rainfall (['ANNUAL'] attribute) in mm by using the monthly rainfall attributes ['Jan-Feb','Mar-May','Jun-Sep','Oct-Dec'] with the help of 'train_test_split' from sklearn module.

X-Attribute: ['Jan-Feb','Mar-May','Jun-Sep','Oct-Dec']

Y-Attribute: ['ANNUAL']

```
from sklearn import linear_model
print("__Multiple Linear regression model between annual rainfall and the periodic rainfall__")
y=data['ANNUAL']
x=data[['Jan-Feb','Mar-May','Jun-Sep','Oct-Dec']]
train_x,test_x,train_y,test_y=train_test_split(x,y,test_size=0.3,shuffle=False)
'''train_x=train_x[:,np.newaxis]
test_x=test_x[:,np.newaxis]'''
print("Train x shape",train_x.shape,"; Test_x",test_x.shape)
print("Train y shape",train_y.shape,"; Test_y",test_y.shape)
lm=linear_model.LinearRegression()
lm.fit(train_x,train_y)
pred=lm.predict(test_x)
#print(test_y)
#print(pred)
print("Mean Squared Error =",mean_squared_error(test_y,pred))
print("Root Mean Squared Error =",np.sqrt(mean_squared_error(test_y,pred)))
print("Mean Absolute Error =",mean_absolute_error(test_y,pred))
print("r2_score =",r2_score(test_y,pred))
plt.scatter(pred,test_y)
plt.xlabel('TRAIN_X')
plt.ylabel('TRAIN_Y')
plt.show()
```

TESTING THE MODEL USING PERFORMANCE MEASURES:

There are various measures to test the accuracy of our model like:

MEAN SQUARED ERROR:

If $\hat{Y}$ is a vector of **n** predictions generated from a sample of *n* data points on all variables, and Y is the vector of observed values of the variable being predicted, then the within-sample MSE of the predictor is computed as

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

I.e., the MSE is the *mean* $\left(\frac{1}{n}\sum_{i=1}^{n}\right)$ of the *squares of the errors* $(Y_i - \hat{Y}_i)^2$. This is an easily computable quantity for a particular sample (and hence is sample-dependent).

The MSE can also be computed on *q* data points that were not used in estimating the model, either because they were held back for this purpose or because these data have been newly obtained. In this process, which is known as cross-validation, the MSE is often called the mean squared prediction error, and is computed as

$$\mathrm{MSPE} = \frac{1}{q}\sum_{i=n+1}^{n+q}(Y_i - \hat{Y}_i)^2.$$

ROOT MEAN SQUARED ERROR:

Root Mean Square Error (RMSE) is the standard deviation of the residuals (prediction errors). Residuals are a measure of how far from the regression line data points are; RMSE is a measure of how spread out these residuals are. In other words, it tells you how concentrated the data is around the line of best fit. Root mean square error is commonly used in climatology, forecasting, and regression analysis to verify experimental results.

The formula is:

$$RMSE = \sqrt{\overline{(f-o)^2}}$$

Where:

- f = forecasts (expected values or unknown results),
- o = observed values (known results).

The bar above the squared differences is the mean (similar to $\bar{x}$). The same formula can be written with the following, slightly different, notation (Barnston, 1992):

$$\mathrm{RMSE}_{fo} = [\sum_{i=1}^{N} (z_{f_i} - z_{o_i})^2 / N]^{1/2}$$

Where:

- Σ = summation ("add up")
- $(z_{fi} - Z_{oi})$Sup>2 = differences, squared
- N = sample size.

MEAN ABSOLUTE ERROR:

mean absolute error (MAE) is a measure of difference between two continuous variables. Assume *X* and *Y* are variables of paired observations that express the same phenomenon. Examples of *Y* versus *X* include comparisons of predicted versus observed, subsequent time versus initial time, and one technique of measurement versus an alternative technique of measurement. Consider a scatter plot of *n* points, where point *i* has coordinates (x_i, y_i)... Mean Absolute Error (MAE) is the average vertical distance between each point and the identity line. MAE is also the average horizontal distance between each point and the identity line.

The Mean Absolute Error is given by:

$$\text{MAE} = \frac{\sum_{i=1}^{n} |y_i - x_i|}{n} = \frac{\sum_{i=1}^{n} |e_i|}{n}$$

R SQUARED ERROR:

R-squared is given by

$$R^2 = 1 - \frac{SSE}{SST}$$

Where **SSE is the sum of squared errors of our regression model**

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

And **SST is the sum of squared errors of our baseline model.**

$$SST = \sum_{i=1}^{n} (y_i - \bar{y}_i)^2$$

The following results were obtained:

```
__Multiple Linear regression model between annual rainfall and the periodic rainfall__
Train x shape (2881, 4) ; Test_x (1235, 4)
Train y shape (2881,) ; Test_y (1235,)
Mean Squared Error = 3326.4157535418863
Root Mean Squared Error = 57.67508780697162
Mean Absolute Error = 10.953757241508946
r2_score = 0.9958637383726687
```

Mean Squared Error:

3326.4157535418863

Root Mean Squared Error:

57.67508780697162

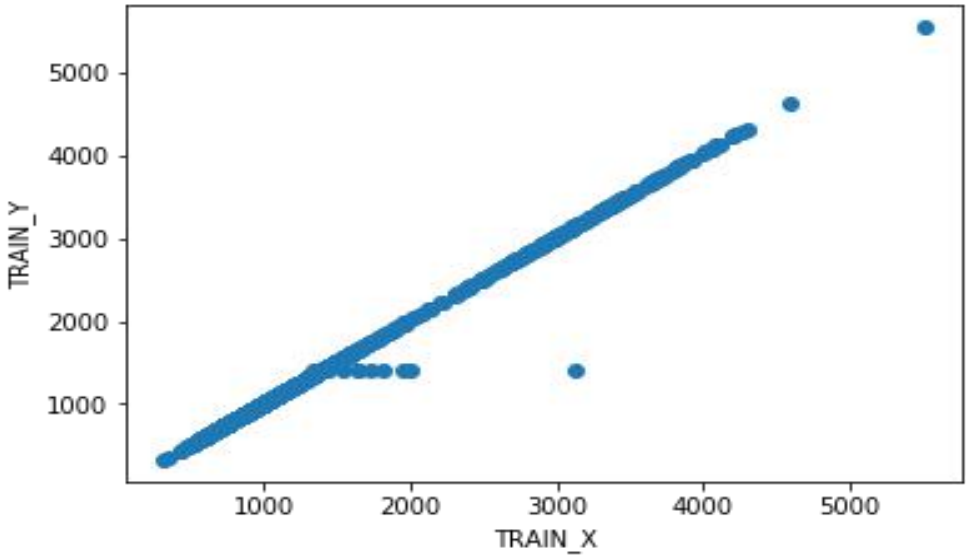
Mean Absolute Error:

10.953757241508946

r2_score:

0.9958637383726687

SCATTER PLOT OF TRAIN_X AND TRAIN_Y (PREDICTION):



JUSTIFICATION OF FITTED MODEL

From the above scatter plot, we can clearly see that the relation between train and test models is a straight line, also the expected and predicted values of annual rainfall are obtained. Let us observe the first 3 and last 3 values of expected and predicted annual rainfall, rounding off to one decimal places.

EXPECTED VALUE:

983.1, 1404.2, 1130.5 ... 1426.3, 1395.0, 1642.9

PREDICTED VALUE:

984.2, 1400.5, 1128.3 ... 1420.5, 1389.3, 1639.9

From the above results we can observe that the expected and predicted values of annual rainfall have relatively less difference. Thus we can say that our prediction is good. But let us proceed further to examine the accuracy of our result with the help of other tools such as accuracy score , classification report.

CLASSIFICATION METRICS

We can calculate the accuracy of the prediction and produce a classification report with the help of functions such as accuracy_score, classification_report ,confusion_matrix from the sklearn module.

1. Classification Accuracy

Classification accuracy is the number of correct predictions made as a ratio of all predictions made.

This is the most common evaluation metric for classification problems, it is also the most misused. It is really only suitable when there are an equal number of observations in each class (which is rarely the case) and that all predictions and prediction errors are equally important, which is often not the case.

2. Classification Report

Scikit-learn does provide a convenience report when working on classification problems to give you a quick idea of the accuracy of a model using a number of measures.

The *classification_report()* function displays the precision, recall, f1-score and support for each class.

3. Confusion Matrix

The confusion matrix is a handy presentation of the accuracy of a model with two or more classes.

The table presents predictions on the x-axis and accuracy outcomes on the y-axis. The cells of the table are the number of predictions made by a machine learning algorithm.

For example, a machine learning algorithm can predict 0 or 1 and each prediction may actually have been a 0 or 1. Predictions for 0 that were actually 0 appear in the cell for prediction=0 and actual=0, whereas predictions for 0 that were actually 1 appear in the cell for prediction = 0 and actual=1. And so on.

Since our data is numeric , we classify the annual rainfall into two types ‘low’ and ‘high’. If the annual rainfall is >2000 cm it is classified as ‘high’ rainfall else it is labelled as ‘low’ rainfall.

This process is done on both the expected and predicted values obtained.

```
expected=[]
for i in test_y:
    if i>2000:
        expected.append("high")
    else:
        expected.append("low")
predicted=[]
for i in pred:
    if i>2000:
        predicted.append("high")
    else:
        predicted.append("low")
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report
acc=accuracy_score(predicted,expected)
matrix=confusion_matrix(predicted,expected)
clas=classification_report(predicted,expected)
print("accuracy")
print(acc)
print("\n")
print("classification")
print(clas)
```

Then by using the above functions we get the following results.

High rainfall is marked as 1 and low rainfall as 0.

```
accuracy
0.745748987854251


classification report
             precision    recall  f1-score   support

       high       0.17      0.13      0.15       207
        low       0.83      0.87      0.85      1028

avg / total       0.72      0.75      0.73      1235

confusion matrix

[[ 27 180]
 [134 894]]
```

ROC CURVE

It is a graph showing the performance of a classification model at all classification thresholds This curve plots two parameters:

True Positive Rate and **False Positive Rate.**

$$tpr = \frac{tp}{tp+fn}$$

$$fpr = fp/(fp + tn)$$

AUC: AUC stands for Area under curve.

The AUC represents a model's ability to discriminate between positive and negative classes. An area of 1.0 represents a model that made all predictions perfectly. An area of 0.5 represents a model as good as random. Learn more about ROC here.

ROC can be broken down into sensitivity and specificity. A binary classification problem is really a trade-off between sensitivity and specificity.

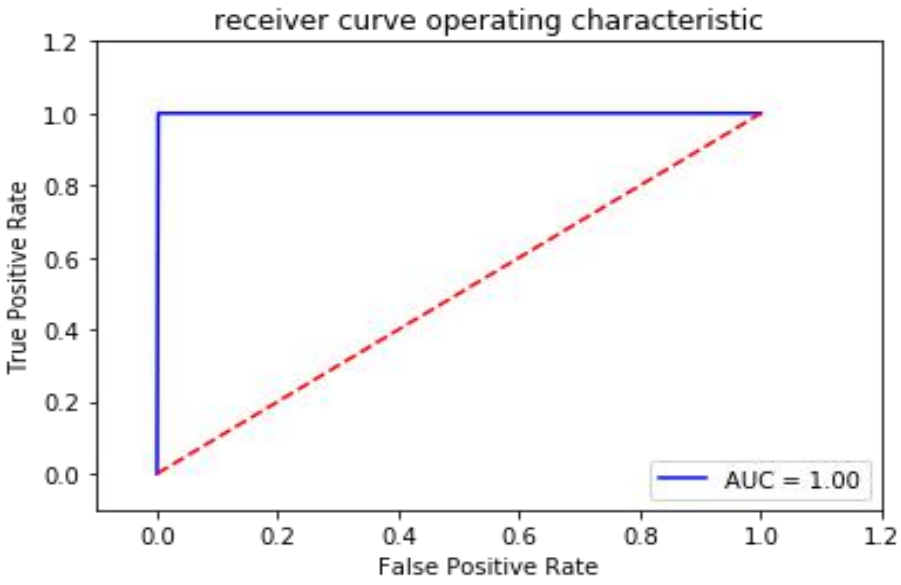
Sensitivity is the true positive rate also called the recall. It is the number instances from the positive (first) class that actually predicted correctly.Specificity is also called the true negative rate. Is the number of instances from the negative class (second)

class that were actually predicted correctly.

```
exp=[]
pre=[]
for i in expected:
    if i=='high':
        exp.append(1)
    else:
        exp.append(0)
for i in predicted:
    if i=='high':
        pre.append(1)
    else:
        pre.append(0)
```

```
from sklearn.metrics import roc_curve,auc
import random
fpr,tpr,threshold=roc_curve(exp,pre)
roc_auc=auc(fpr,tpr)
plt.title("receiver curve operating characteristic")
plt.plot(fpr,tpr,'b',label='AUC = %0.2f'%roc_auc)
plt.legend(loc='lower right')
plt.plot([0,1],[0,1],'r--')
plt.xlim([-0.1,1.2])
plt.ylim([-0.1,1.2])
plt.ylabel("True Positive Rate")
plt.xlabel("False Positive Rate")
plt.show()
```

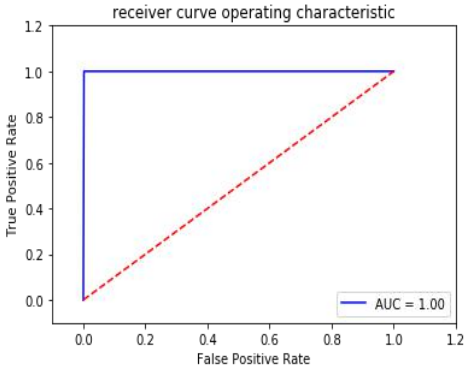
'High' rainfall is marked as '1' and 'low' as '0' to get the desired result.

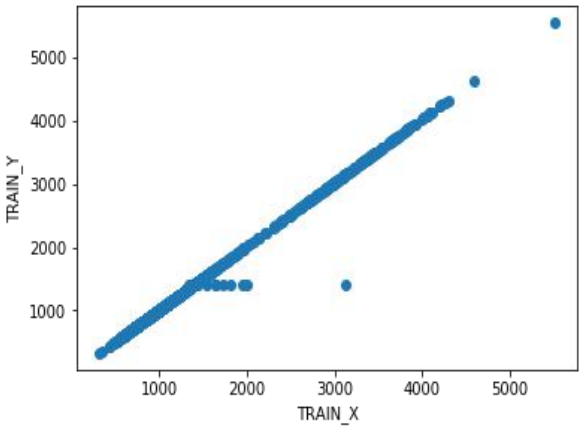

receiver curve operating characteristic
True Positive Rate
False Positive Rate
AUC = 1.00
0.0
0.2
0.4
0.6
0.8
1.0
1.2

DISCUSSION ON RESULTS

We have tested out model in various ways with various performance measures, let us discuss the validity of the results :

```
__Multiple Linear regression model between annual rainfall and the periodic rainfall__
Train x shape (2881, 4) ; Test_x (1235, 4)
Train y shape (2881,) ; Test_y (1235,)
Mean Squared Error = 3326.4157535418863
Root Mean Squared Error = 57.67508780697162
Mean Absolute Error = 10.953757241508946
r2_score = 0.9958637383726687
```






```
accuracy
0.745748987854251


classification report
             precision    recall  f1-score   support

       high       0.17      0.13      0.15       207
        low       0.83      0.87      0.85      1028

avg / total       0.72      0.75      0.73      1235

confusion matrix

[[ 27 180]
 [134 894]]
```

PERFORMANCE MEASURES:

RMSE- Our Expected and predicted values are in the range of >1000 mm relative to which our obtained rmse of 57.65 is relatively less. Lesser the value of rmse better is the accuracy of the prediction.

MEAN ABSOLUTE ERROR: similarly we have obtained a mean absolute error of 10.95 which is a very small error.

R-SQUARED : R-squared is a statistical measure of how close the data are to the fitted regression line. It is also known as the coefficient of determination, or the coefficient of multiple determination for multiple regression. A coefficient closer to one indicates that the model explains all the variability of the response data around its mean. We have obtained a r-squared value of 0.99 using the attribute r2_score which is a very good result.

SCATTER PLOT OF RESULT (TRAIN_X VS TRAIN_Y)

As we can see the scatter plot of our prediction is almost a straight line i.e the data is very concentrated This indicates that our prediction is good

CLASSIFICATION ACCURACY, REPORT AND CONFUSION MATRIX

Using the attribute accuracy_score we have obtained an accuracy of approximately 0.745 which can be converted into a percentage of 74.5% which is a decent prediction.

Also by using classification report we obtained the precision of the model in two categories ‘high’ and ‘low’ rainfall. The obtained results are:

High rainfall – 17%

Low rainfall - 83%

The confusion matrix tells us that we obtained a

True positive: 27

False negative: 180

False positive: 134

True negative: 894

CONCLUSION AND FUTURE WORKS

We have observed that the annual rainfall can be predicted directly from monthly rainfall in various regions over a span of 100 years by using multiple linear regression.This shows us that multiple linear regression is a good tool for rainfall prediction and weather forecasting.By being able to predict the amount of rainfall and also distinguish high and low rainfall,with our model we can:

- Predict the rainfall in future years based on the past historical data.
- Predict the variation in the rainfall in future years based on the other atmospheric parameters such as temperature,humidity, atmospheric content(O2,CO2 etc.).
- Predict the time and area of rainfall in a particular region.
- Predict natural calamities due to rainfall such as floods,droughts etc.
- Determine the effect of man-made calamities such as global warming and deforestation on rainfall.
- Predict the chance of acid of acid rain based on the pollution in the state.

Provided if we are given a better and bigger data set with the relevant data and less limitations.

LIST OF REFERENCES:

WEBSITES :

- **Machine-learning-pdfs-https://teleuniv.net/Catalog/kmit/filebrowser/index.php**
- **Dataset-https://www.kaggle.com/rajanand/rainfall-in-india#rainfall%20in%20india%201901-2015.csv-**
- **Multi-linear-regression-https://rspublication.com/ijca/2016/april16/5.pdf**
- **Metrics-https://machinelearningmastery.com/metrics-evaluate-machine-learning-algorithms-python/**

BOOKS:

- **Hands on machine learning with sci-kit learn and tensor flow-https://universalflowuniversity.com/Books/Computer%20Programming/Machine%20Learning%20and%20Deep%20Learning/Hands-On%20Machine%20Learning%20with%20Scikit-Learn%20%26%20TensorFlow.pdf**
- **Numpy-and-pandas-using-python-http://download.library1.org/main/864000/ae0913ccbd281d74c48829644dc8**

a8d9/Wes%20McKinney-Python%20for%20Data%20Analysis-O%27Reilly%20Media%20%282012%29.pdf